

SysY Language Definition (2022 Edition)

1. Overview of the SysY Language

SysY is the programming language to be implemented in the Compiler System Design Competition. It is extended from a subset of the C language. The source code of each SysY program is stored in a file whose extension is `.sy`. Such a file must contain exactly one definition of the `main` function, and may also contain several global variable declarations, constant declarations, and other function definitions. SysY supports the types `int` and `float`, as well as multidimensional array types whose element type is `int` or `float` and whose storage order is row-major. Here, `int` is a 32-bit signed integer, `float` is a 32-bit single-precision floating-point number, and the `const` qualifier is used to declare constants. SysY supports implicit conversions between `int` and `float`, but does not support explicit cast syntax.¹

SysY itself does not provide built-in language constructs for input/output (I/O). I/O is provided by way of a runtime library, and library functions may be called inside functions in SysY programs. Some SysY runtime-library functions accept parameter types beyond the data types supported by SysY itself, such as strings. A SysY compiler must be able to handle such cases and correctly pass such arguments from a SysY program to the SysY runtime library. For the library functions available in SysY programs, see the SysY runtime-library documentation.

Functions: A function may have parameters or may have none. Parameter types may be `int`, `float`, or array types. A function may return a value of type `int` or `float`, or may return no value (that is, it may be declared with type `void`). Parameters of type `int/float` are passed by value; parameters of array type are passed by passing the starting address of the array, and only the first dimension of the formal parameter may omit its length. A function body consists of several variable declarations and statements.

Variable/constant declarations: A single declaration statement may declare multiple variables or constants, and initialization expressions may be provided in the declaration. All variables/constants must be defined before use. Declarations outside functions are global variables/constants, while declarations inside functions are local variables/constants.

Statements: Statements include assignment statements, expression statements (where the expression may be empty), statement blocks, `if` statements, `while` statements, `break` statements, `continue` statements, and `return` statements. A statement block may contain several variable declarations and statements.

Expressions: SysY supports basic arithmetic operations (`+`, `-`, `*`, `/`, `%`), relational operations (`==`, `!=`, `<`, `>`, `<=`, `>=`), and logical operations (`!`, `&&`, `||`). A nonzero value means true, and 0 means false; the result of a relational or logical operation is represented as 1 for true and 0 for false. Operator precedence, associativity, and evaluation rules, including short-circuit evaluation for logical operations, are the same as in C.

¹Compared with the 2020 edition, SysY2022 adds the basic type `float` and supports multidimensional arrays whose element type is `float`.

2. Grammar of the SysY Language

The grammar of SysY is written in Extended Backus-Naur Form (EBNF), where:

- [...] denotes an optional item.
- {...} denotes an item that may be repeated zero or more times.
- A terminal symbol is either a quoted string or a token such as `Ident`, `IntConst`, or `floatConst`.

The grammar of SysY is as follows. `CompUnit` is the start symbol:

```
CompUnit    -> [ CompUnit ] ( Decl | FuncDef )
Decl        -> ConstDecl | VarDecl
ConstDecl   -> 'const' BType ConstDef { ',' ConstDef } ';'
BType       -> 'int' | 'float'
ConstDef    -> Ident { '[' ConstExp ']' } '=' ConstInitVal
ConstInitVal -> ConstExp
              | '{' [ ConstInitVal { ',' ConstInitVal } ] '}'

VarDecl     -> BType VarDef { ',' VarDef } ';'
VarDef      -> Ident { '[' ConstExp ']' }
              | Ident { '[' ConstExp ']' } '=' InitVal
InitVal     -> Exp | '{' [ InitVal { ',' InitVal } ] '}'

FuncDef     -> FuncType Ident '(' [FuncFParams] ')' Block
FuncType    -> 'void' | 'int' | 'float'
FuncFParams -> FuncFParam { ',' FuncFParam }
FuncFParam  -> BType Ident '[' ']' { '[' Exp ']' }

Block       -> '{' { BlockItem } '}'
BlockItem   -> Decl | Stmt
Stmt        -> LVal '=' Exp ';'
              | [Exp] ';'
              | Block
              | 'if' '(' Cond ')' Stmt [ 'else' Stmt ]
              | 'while' '(' Cond ')' Stmt
              | 'break' ';'
              | 'continue' ';'
              | 'return' [Exp] ';'

Exp         -> AddExp
Cond        -> LOrExp
LVal        -> Ident { '[' Exp ']' }
PrimaryExp  -> '(' Exp ')' | LVal | Number
Number      -> IntConst | floatConst
UnaryExp    -> PrimaryExp | Ident '(' [FuncRParams] ')'
              | UnaryOp UnaryExp
UnaryOp     -> '+' | '-' | '!'
FuncRParams -> Exp { ',' Exp }
MulExp      -> UnaryExp | MulExp ('*' | '/' | '%') UnaryExp
```

```

AddExp      -> MulExp | AddExp ('+' | '-') MulExp
RelExp      -> AddExp | RelExp ('<' | '>' | '<=' | '>=') AddExp
EqExp       -> RelExp | EqExp ('==' | '!=') RelExp
LAndExp     -> EqExp | LAndExp '&&' EqExp
LOrExp      -> LAndExp | LOrExp '||' LAndExp
ConstExp    -> AddExp

```

Notes:

- Exp in SysY is an expression of type `int/float`.
- ! appears only in conditional expressions.
- Any `Ident` used in `ConstExp` must denote a constant.

Terminal Symbols in SysY

1. Identifier `Ident` The syntax of identifiers `Ident` in SysY is as follows (`identifier`):

```

identifier -> identifier-nondigit
            | identifier identifier-nondigit
            | identifier digit

```

Here, `identifier-nondigit` is `_` or one of the following:

```

a b c d e f g h i j k l m
n o p q r s t u v w x y z
A B C D E F G H I J K L M
N O P Q R S T U V W X Y Z

```

`digit` is one of the following:

```

0 1 2 3 4 5 6 7 8 9

```

Note: See the definition of identifiers on page 51 of ISO/IEC 9899:
<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1124.pdf>

Conventions for identically named identifiers:

- The scopes of global variables and local variables may overlap; in the overlapping region, the local variable takes precedence.
- The scopes of local variables with the same name may not overlap.
- In SysY, a variable name may be the same as a function name.

2. Comments The syntax of comments in SysY is the same as in C:

- Single-line comments start with the sequence `//` and end at the newline character; the newline itself is not part of the comment.
- Multi-line comments start with the sequence `/*` and end at the first occurrence of `*/`, including the closing `*/`.

Note: See the definition of comments on page 66 of ISO/IEC 9899:
<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1124.pdf>

3. Numeric Constants Numeric constants in SysY may be integer constants `IntConst` or floating-point constants `FloatConst`.

The syntax of integer constants `IntConst` is as follows (corresponding to `integer-const`):

```
integer-const      -> decimal-const | octal-const
                   | hexadecimal-const
decimal-const      -> nonzero-digit | decimal-const digit
octal-const        -> 0 | octal-const octal-digit
hexadecimal-const  -> hexadecimal-prefix hexadecimal-digit
                   | hexadecimal-const hexadecimal-digit
```

```
hexadecimal-prefix -> '0x' | '0X'
```

`nonzero-digit` is one of:

1 2 3 4 5 6 7 8 9

`octal-digit` is one of:

0 1 2 3 4 5 6 7

`hexadecimal-digit` is one of:

0 1 2 3 4 5 6 7 8 9 a b c d e f A B C D E F

Note: See the definition of integer constants on page 54 of ISO/IEC 9899: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1124.pdf> On that basis, all suffixes are ignored here.

The definition of floating-point constants `FloatConst` refers to the definition of floating constants on page 57 of ISO/IEC 9899: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1124.pdf> On that basis, all suffixes are ignored here as well.

3. Semantic Constraints of the SysY Language

The set of programs matching the grammar above is a superset of the set of legal SysY programs. The semantic constraints of SysY are further specified below.

CompUnit

```
CompUnit -> [ CompUnit ] ( Decl | FuncDef )
Decl      -> ConstDecl | VarDecl
```

1. A SysY program consists of a single file whose contents correspond to `CompUnit` in the EBNF above. In that `CompUnit`, there must exist exactly one `FuncDef` whose identifier is `main`, whose parameter list is empty, and whose return type is `int`. The `main` function is the entry point of the program, and the return value of `main` must be output.
2. At the top level of `CompUnit`, variable/constant declaration statements (`Decl`) and function definitions (`FuncDef`) may not redefine the same identifier (`Ident`), even if the types differ.
3. The scope of a variable/constant/function declaration in `CompUnit` starts at the point of declaration and extends to the end of the file.

ConstDef

`ConstDef -> Ident { '[' ConstExp ']' } '=' ConstInitVal`

1. `ConstDef` defines a symbolic constant. In `ConstDef`, `Ident` is the identifier of the constant. The part after `Ident` and before `=` optionally defines the array dimensions and the length of each dimension, and the part after `=` is the initializer.
2. If the array-dimension part of `ConstDef` is absent, a single variable is defined. In this case, the right-hand side of `=` must be a single initial numeric value.
3. If the array-dimension part of `ConstDef` is present, an array is defined. Its semantics are the same as in C. For example, `[2][8/2][1*3]` denotes a three-dimensional array whose first through third dimensions have lengths 2, 4, and 3, respectively, and whose indices start from 0 in each dimension. Every `ConstExp` used to specify a dimension length in `ConstDef` must be evaluable at compile time to a nonnegative integer.
4. SysY requires every array dimension length in a declaration to be given explicitly; unknown dimension sizes are not allowed.
5. When `ConstDef` defines an array, the `ConstInitVal` on the right-hand side of `=` is a constant initializer. The `ConstExp` expressions that appear in `ConstInitVal` are `int/float` expressions evaluable at compile time, and may refer to already-defined symbolic constants.
6. A `ConstInitVal` initializer must be one of the following:
 1. A pair of braces `{}`, meaning all elements are initialized to 0.
 2. An initializer that fully corresponds to the array rank and each dimension length of the multidimensional array. For example, `{{1,2},{3,4},{5,6}}`, `{1,2,3,4,5,6}`, and `{1,2,{3,4},5,6}` may all be used as initializers for `a[3][2]`.
 3. If the number of initializers in a brace-enclosed list is smaller than the number of elements in the corresponding dimension, the remaining part of that dimension is implicitly initialized. Every implicitly initialized integer element is initialized to 0. For example, `{{1, 2},{3}, {5}}`, `{1,2,{3},5}`, and `{{},{3,4},5,6}` may all be used as initializers for `a[3][2]`. The first two initialize `a` as `{{1,2},{3,0},{5,0}}`, while `{{},{3,4},5,6}` initializes `a` as `{{0,0},{3,4},{5,6}}`.
7. For example, the declarations and initializations of variables `a` through `e` below are all allowed:

```
int a[4][2] = {};  
int b[4][2] = {1, 2, 3, 4, 5, 6, 7, 8};  
int c[4][2] = {{1, 2}, {3, 4}, 5, 6, 7, 8};  
int d[4][2] = {1, 2, {3}, {5}, 7, 8};  
int e[4][2] = {{1, 2}, {3, 4}, {5, 6}, {7, 8}};
```

8. The type of each array-element initializer must be consistent with the declared element type of the array. For example, the initializer list of an integer array may not contain floating-point elements; however, the initializer list of a floating-point array may contain integer constants or integer constant expressions.

9. The value of each array-element initializer may not exceed the representable range of the corresponding element type.
10. The number of elements in the initializer list may not exceed the total number of elements specified in the array declaration.

VarDef

```
VarDef -> Ident { '[' ConstExp ']' }
        | Ident { '[' ConstExp ']' } '=' InitVal
```

1. **VarDef** defines a variable. If = and an initializer are absent, the actual runtime initial value is undefined.
2. If the array-dimension part of **VarDef** is absent, a single variable is defined. If it is present, then, as in **ConstDef**, a multidimensional array is defined.
3. When **VarDef** contains = and an initializer, the structure of the **InitVal** on the right-hand side of = must be the same as that of **ConstInitVal**. The only difference is that the expressions in **ConstInitVal** are constant expressions **ConstExp**, while the expressions in **InitVal** may be any **Exp** valid in the current context.
4. Each **ConstExp** used to specify a dimension length in **VarDef** must be evaluable to a nonnegative integer, but the initializer values in **InitVal** are **Exp** and may therefore refer to variables. For example, in the code below, the initializer expression of variable **e** uses **d[2][1]**.

```
int a[4][2] = {};
int b[4][2] = {1, 2, 3, 4, 5, 6, 7, 8};
int c[4][2] = {{1, 2}, {3, 4}, 5, 6, 7, 8};
int d[4][2] = {1, 2, {3}, {5}, 7, 8};
int e[4][2] = {{d[2][1], c[2][1]}, {3, 4}, {5, 6}, {7, 8}};
```

Initial Values

```
ConstInitVal -> ConstExp
               | '{' [ ConstInitVal { ',' ConstInitVal } ] '}'
```

```
InitVal      -> Exp
               | '{' [ InitVal { ',' InitVal } ] '}'
```

1. In a global-variable declaration, the specified initializer expression must be a constant expression.
2. The initializer specified in a constant or variable declaration must be consistent with the type of that constant or variable. The following forms of **VarDef** / **ConstDef** do not satisfy the SysY semantic constraints:

```
a[4] = 4
a[2] = {{1,2}, 3}
a = {1,2,3}
```

3. A local variable that is not explicitly initialized has an indeterminate value; a global variable that is not explicitly initialized has its value (or its elements) initialized to 0 or 0.0.

FuncFParam and Actual Arguments

FuncFParam -> BType Ident '[' ']' { '[' Exp ']' }

1. **FuncFParam** defines one formal parameter of a function. When the optional part after **Ident** is present, it denotes an array definition.
2. When **FuncFParam** is an array definition, the first-dimension length is omitted (written as `[]`), while each subsequent dimension must be given by an expression whose value is an integer constant.
3. The syntax of actual arguments is **Exp**. For parameters of type `int/float`, arguments are passed by value. For parameters of array type, the formal parameter receives the address of the actual argument array and accesses the actual array elements indirectly through that address.
4. For a multidimensional array, part of it may be passed to an array formal parameter. For example, if `int a[4][3]`, then `a[1]` is a one-dimensional array with three elements, and `a[1]` may be passed as an argument to a formal parameter of type `int[]`.

FuncDef

FuncDef -> FuncType Ident '(' [FuncFParams] ')' Block

1. **FuncDef** denotes a function definition. **FuncType** specifies the return type.
 1. When the return type is `int/float`, every branch of the function should contain a **return** statement with an **Exp**. The return value of a branch that contains no **return** statement is undefined.
 2. When the return type is `void`, only **return** statements without a return value may appear in the function.
2. Each formal parameter declaration (**FuncFParam**) in the formal-parameter list (**FuncFParams**) of **FuncDef** declares either an `int/float` parameter or a multidimensional array parameter whose element type is `int/float`. The semantics of **FuncFParam** are given above.

Block

1. **Block** denotes a statement block. A statement block creates a scope, and the lifetime of variables declared in the block is limited to that block.
2. Inside a statement block, variables or constants with the same name as declarations outside the block may be redefined (through **Decl**). Their scope begins at the point of definition and ends at the end of the block, and they hide the outer declarations with the same name.

Stmt

1. **if** statements in **Stmt** follow the nearest-**else** matching rule.
2. A single **Exp** may appear as a **Stmt**. The expression is evaluated and the resulting value is discarded.

LVal

1. **LVal** denotes an expression that has an lvalue; it may be a variable or an array element.

2. When **LVal** denotes an array element, the number of bracket pairs must equal the number of dimensions of the array variable; that is, it must identify an element.
3. When **LVal** denotes a single variable, no following brackets may appear.

Exp and Cond

1. In SysY, **Exp** denotes an expression of type **int/float**, so it is defined as **AddExp**; **Cond** denotes a conditional expression, so it is defined as **LOrExp**. The former therefore does not include **!** among its unary operators, while the latter may include it. In addition, when **Exp** is used as an array dimension, it must evaluate to a nonnegative integer.
2. **LVal** must denote a variable or constant that is defined in the current scope before the **Exp** statement in which it appears. For the **LVal** on the left-hand side of an assignment, it must denote a variable.
3. A function call has the form **Ident '(' FuncRParams ')'**, where **FuncRParams** denotes the actual arguments. The types and number of actual arguments must exactly match the formal parameters of the function defined by **Ident**.
4. Operator precedence and associativity in SysY are the same as in C, and the grammar above already reflects those precedence and associativity rules.

Implicit Type Conversions

The following implicit type conversions exist between values of type **int** and **float** in SysY:

1. When a value of type **float** is implicitly converted to an integer, for example by **int i = 4.0;**, the fractional part is discarded. If the integral part is outside the representable range of the integer type, the behavior is undefined.
2. When a value of type **int** is converted to **float**, for example by **float j = 3;**, the converted value remains unchanged.

Note: When implementing implicit type conversions, a compiler needs to combine the type-conversion instructions provided by the hardware architecture with the runtime ABI (Application Binary Interface). For example, on ARM, the runtime ABI function **float __aeabi_i2f(int)** may be called to convert an **int** to a **float**.

See: <https://developer.arm.com/documentation/ih0043/latest>